

EEE 598 Assignment 2: Perceptual Losses, ViT Fine-Tuning for PV Fault Classification, and Custom ResNets on ImageNet

Qitong [TODO: last name]

Arizona State University

qitongzh@asu.edu

Abstract

This report covers the three problems of Assignment 2. (1) Perceptual losses. A 1-pixel translation of a 4032×3024 photo changes the pixels by only 9.0% (relative ℓ_2), but changes VGG-16 features by 32–59%. Controlled experiments trace this to aliasing in the strided and max-pooled layers; spatially pooled feature statistics change by at most 3.8%. Under additive noise, deep features (`relu3_3`, `relu4_3`) saturate while shallow ones keep growing. A turbulence-robust feature built from these findings (an anti-aliased VGG trunk with a multi-scale head trained by a dense contrastive loss on DAATSim data) is far more robust than VGG features under weak turbulence, but does not beat a simple blurred-pixel baseline; we analyze why. (2) PV faults. We balance the 12-class infrared PV dataset to 2,500 images per class with unsharp masking and flip/jitter augmentation (20,000 $\rightarrow$ 30,000 images), and fine-tune ViT-B/32 with three strategies on a leakage-free split. Full fine-tuning is the most accurate (73.3% test, 4.2 min); the last 4 blocks reach 68.7% in 2.3 min; the head alone reaches only 56.3%. (3) ImageNet. On Sol's 650-class ImageNet we design ResNet-36 (one extra stage-3 block) and a new activation, ArcGate $f(x) = x(\frac{1}{2} + \frac{1}{\pi} \arctan x)$. On a 100-class prototype, ArcGate matches ReLU in top-1, improves top-5 by 1.1 points and shrinks the train-test gap by 2.7 points (two seeds). Trained on the full dataset for 10 epochs, ResNet-36+ArcGate reaches 60.4% / 82.9% test top-1/top-5 on $2 \times A100$ and 60.3% / 82.8% on $2 \times$ Gaudi 2, where training is 13% faster. It clearly outperforms a ViT-S/16 trained from scratch (45.4% top-1).

1. Overview and computing setup

All experiments ran on ASU's Sol supercomputer [2]. Problems 1 and 2 ran in Jupyter sessions on A100 MIG slices (2g.20gb). Problem 3 ran as SBATCH jobs on full A100-SXM4-80GB GPUs (partition `public`, QOS `class`) and on Intel Gaudi 2 (HL-225) accelerators (partition `gaudi`,

Problem	Quantity	Result
1.1	1-px shift: mean ℓ_1 / MSE (pixels in $[0,1]$)	0.0248 / 0.00217
1.2	1-px shift: rel. change, pixels vs. <code>relu1_2</code> .. <code>relu4_3</code>	0.090 vs. 0.59..0.32
1.3	rel. change at $\sigma=0.3$: <code>relu1_2</code> / <code>relu4_3</code>	1.96 / 1.16
1.4	AUC _{edit} at $D/r_0=2$: VGG / ours / blurred pixels	0.00 / 0.76 / 1.00
2.1	PV images before $\rightarrow$ after balancing	20,000 $\rightarrow$ 30,000
2.2	ViT-B/32 test acc.: head / last 4 blocks / full	56.3 / 68.7 / 73.3%
3.2	proto. test top-1, ResNet-34 / ResNet-36 (2 seeds)	62.5 / 63.1%
3.3	proto. test top-5, ReLU / ArcGate (2 seeds)	82.6 / 83.6%
3.4	full test top-1, A100 / Gaudi 2	60.4 / 60.3%
3.4	train time 10 ep., A100 / Gaudi 2 (2 devices)	53.9 / 46.8 min
3.5	full test top-1, ResNet-36+ArcGate / ViT-S/16	60.4 / 45.4%

Table 1. Main results at a glance. Details are in the section for each problem.

QOS `class_gaudi`). All code is PyTorch [14]. Every training run uses `bfloat16` autocast. Tab. 1 lists the main results.

2. Problem 1: Perceptual losses for images

2.1. Pixel-space losses of a 1-pixel translation

We use a 4032×3024 smartphone photo of Daocheng Yading, China (Fig. 1), which is above the required 1920×1080 . The image is loaded as RGB and scaled to $[0, 1]$. Translating it one pixel to the left gives $x'_{i,j} = x_{i,j+1}$; the right-most column has nothing to shift in and is set to 0 (Listing 1). Over all $N = 3024 \times 4032 \times 3 = 36.6$ M values, the losses are

$$\ell_1 : \sum |x' - x| = 906,791.4, \quad \frac{1}{N} \sum |x' - x| = 0.024790, \quad (1)$$

$$\ell_2 : \sum (x' - x)^2 = 79,364.7, \quad \frac{1}{N} \sum (x' - x)^2 = 0.002170, \quad (2)$$

and $\|x' - x\|_2 = 281.72$. Fig. 1 shows the original and the translated image.

Answer. $\ell_1 = 906,791$ (sum) or 0.0248 (mean); $\ell_2 = 79,365$ (sum of squares) or 0.00217 (MSE), with $\|x' - x\|_2 = 281.7$, for pixel values in $[0, 1]$. The MSE equals that of adding Gaussian noise with $\sigma = \sqrt{0.00217} \approx 0.047$, even though the shifted image looks identical to a human.

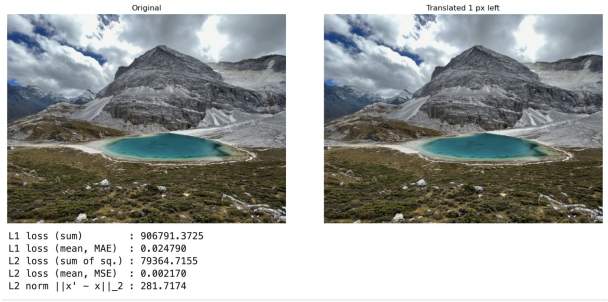


Figure 1. Problem 1.1: the 4032×3024 test image (Daocheng Yading) and its 1-pixel left translation, with the ℓ_1 and ℓ_2 losses printed below. The two images are visually indistinguishable.

```
img = np.asarray(image_pil, np.float64) / 255.0 # [0,1]
shifted = np.zeros_like(img)
shifted[:, :-1] = img[:, 1:] # pixel (r,c) <- (r,c+1)
diff = shifted - img
l1_sum, l1_mean = np.abs(diff).sum(), np.abs(diff).mean()
l2_sum, l2_mean = (diff**2).sum(), (diff**2).mean()
l2_norm = np.sqrt(l2_sum) # ||x' - x||_2
```

Code 1. Shift and pixel losses (excerpt).

Space	Shape	mean ℓ_1	MSE	rel. change
pixels	3×3024×4031	0.0247	0.00212	0.090
relu1_2	64×3024×4031	0.254	0.447	0.590
relu2_2	128×1512×2015	0.378	1.120	0.557
relu3_3	256×756×1007	0.229	0.663	0.379
relu4_3	512×378×503	0.045	0.049	0.317

Table 2. Change caused by a 1-pixel left shift, in pixel space and at the four VGG-16 taps. ℓ_1 and MSE are not comparable across rows (activation scales differ), so we compare the relative change.

2.2. Change in VGG-16 feature space

Feature taps. We use the ImageNet-pretrained VGG-16 [17] from torchvision and read its features after the last ReLU of the first four blocks (Fig. 2): relu1_2, relu2_2, relu3_3 and relu4_3, which are features [3], [8], [15] and [22]. They span low-level edges (stride 1, 64 channels) to object-part detectors (stride 8, 512 channels), and they are the standard choices for perceptual losses [10, 23]. The image is ImageNet-normalized and processed at full resolution. To avoid an artificial edge at the zero-filled column, the original and the shifted image are compared on their common 3024×4031 region; this is why the pixel row of Tab. 2 differs slightly from Sec. 2.1. For each tap ϕ we report ℓ_1 , MSE and the scale-free *relative change* $\|\phi(x') - \phi(x)\|_2 / \|\phi(x)\|_2$. Only the relative change can be compared across layers whose activations have very different magnitudes.

Observations. Tab. 2 and Fig. 3a show that the shift barely changes the pixels (9.0%) but changes every feature map substantially. The change is largest at relu1_2 (59%) and *decreases* with depth to 32% at relu4_3. In relative terms, the features are 3.5–6.6× *more* sensitive to the translation than the pixel losses. This contradicts the common intuition that “deep features are shift-invariant.” We ran four checks to explain it:

- Identical inputs** give exactly zero change at all taps, so the pipeline is correct.
- Aliasing.** Shifting a feature map back by shift/stride cells, possible only when the shift is a multiple of the stride, makes the change vanish (≤ 0.009 , Fig. 3b). The features are equivariant on their own grid. A 1-pixel shift is not a multiple of the stride after pooling, so max-pooling and subsampling sample different pixels and produce a different map [22]. Even relu1_2 (stride 1) changes by 0.61, because its edge maps are compared at the same coordinates, where sharp edges no longer overlap.
- Edges dominate.** Pixel relative change is 0.145 for raw pixels¹, 0.309 after removing the mean, and 0.611 for a high-pass (edge-only) image. The edge-only value matches relu1_2 (0.613): early VGG layers behave like edge detectors, and the pixel loss looks small only because the smooth (DC) content dominates the pixel norm.
- Sparsity and pooling.** The fraction of exactly-zero activations grows from 55% (relu1_2) to 70%, 85% and 92% (relu4_3). When a few sparse peaks move by one cell, both the old and the new location change. Averaging each channel over space (global average pooling) removes this: the channel means change by only 0.05%, 2.2%, 2.5% and 3.8%.

Choice of perceptual feature. For a perceptual loss we would use a channel-normalized, multi-layer distance on relu2_2 and relu3_3 (an LPIPS-style sum [23]), computed after a small anti-aliasing blur. relu2_2 is the layer Johnson *et al.* use for feature reconstruction [10]. It keeps half-resolution detail, which tasks such as super-resolution need. relu3_3 adds mid-level texture and part information and is 32% less shift-sensitive than relu1_2. relu4_3 is the least shift-sensitive tap, but at 1/8 resolution and 92% sparsity it is too coarse to judge fine detail. Our measurements also show that spatial pooling makes VGG statistics almost shift-invariant ($\leq 3.8\%$), so the blur or pooling step is what makes the loss tolerant to imperceptible misalignment.

¹This check ran on a resized copy, hence 0.145 instead of the 0.090 of Tab. 2.

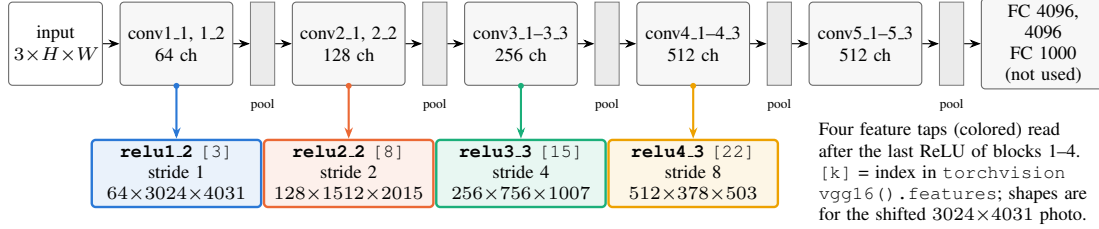


Figure 2. VGG-16 with the four feature-extraction points used in Problems 1.2 and 1.3. Each block is a stack of 3×3 conv + ReLU layers; each max-pool halves the resolution, so the stride doubles from one tap to the next.

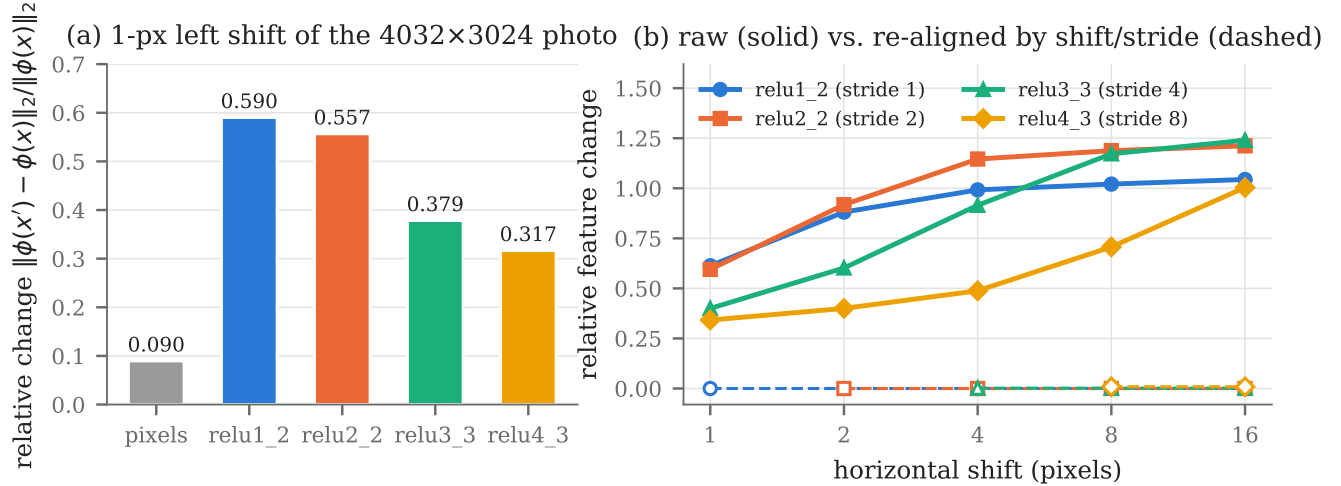


Figure 3. (a) Relative change for a 1-pixel shift: the features change 3.5–6.6× more than the pixels. (b) Verification experiment: relative change vs. shift size (solid). When the shifted feature map is re-aligned by shift/stride cells, which is only possible when the shift is a multiple of the layer’s stride, the change drops to ≤ 0.009 (dashed). The features are therefore exactly shift-equivariant on their own sampling grid, and the 1-pixel change is caused by sampling (aliasing).

Answer. Pixel losses report a small change (9% relative), while raw VGG features report a large one (32–59%) that decreases with depth. The cause is aliasing from stride and max-pooling, not semantics: on an aligned grid the change is $\leq 0.9\%$, and pooled channel statistics change $\leq 3.8\%$. We recommend relu2_2+relu3_3, channel-normalized and slightly blurred or pooled before comparison.

2.3. Robustness to additive noise

We add i.i.d. Gaussian noise with $\sigma \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.3\}$ to the $[0, 1]$ image (PSNR from 40 to 12 dB) and measure the relative change in pixel space and at all four taps. Fig. 4 shows the qualitative result as per-location change maps. Fig. 5 and Tab. 3 show the quantitative result.

Observations. (i) Every feature space is far more sensitive than pixel space: at $\sigma=0.02$ (PSNR 34 dB, barely visible) the pixels change by 3.9% but the features by 22–46%. For relu3_3, this weak noise is as large a change as the 1-pixel shift (0.40 vs. 0.38), while for pixels the noise

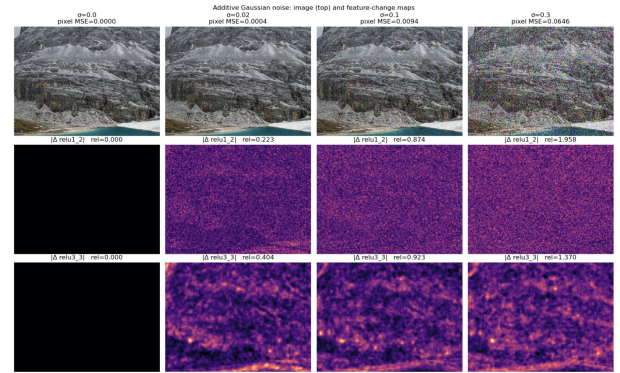


Figure 4. Qualitative noise test. Top: noisy images ($\sigma = 0, 0.02, 0.1, 0.3$). Middle and bottom: per-location feature-change maps $|\Delta|$ for relu1_2 and relu3_3. The relu1_2 change is spatially uniform (it reacts to every noisy pixel). The relu3_3 change concentrates on textured regions such as the rock strata and the lake shore.

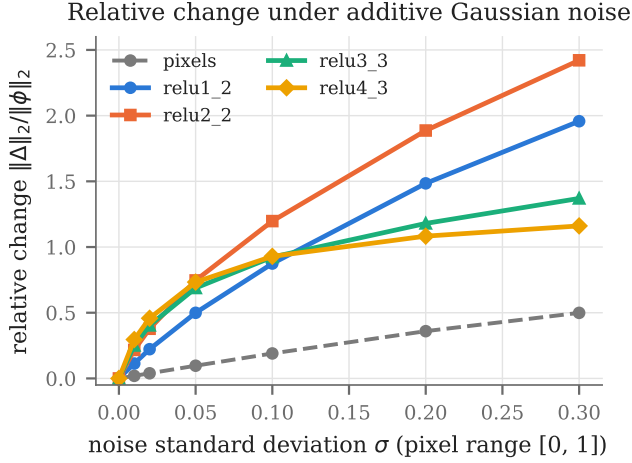


Figure 5. Quantitative noise test: relative change vs. noise level in pixel space and at the four VGG-16 taps. Deeper taps saturate; shallow taps keep growing.

σ	0.01	0.02	0.05	0.1	0.2	0.3
PSNR (dB)	40.0	34.0	26.1	20.3	14.7	11.9
pixel MSE	0.0001	0.0004	0.0024	0.0094	0.0336	0.0646
pixels	0.020	0.039	0.097	0.190	0.360	0.499
relu1_2	0.114	0.223	0.499	0.874	1.485	1.958
relu2_2	0.217	0.378	0.746	1.197	1.887	2.421
relu3_3	0.251	0.404	0.688	0.924	1.180	1.370
relu4_3	0.296	0.458	0.733	0.929	1.083	1.161

Table 3. Relative change $\|\Delta\|_2/\|\phi\|_2$ under additive Gaussian noise. From $\sigma=0.01$ to 0.3 it grows 25 $\times$ for pixels, 17 $\times$ for relu1_2, 11 $\times$ for relu2_2, 5.5 $\times$ for relu3_3 and 3.9 $\times$ for relu4_3.

is less than half the shift (0.039 vs. 0.090). (ii) For weak noise, sensitivity *increases* with depth ($\sigma=0.01$: 0.11, 0.22, 0.25, 0.30 from relu1_2 to relu4_3): faint noise excites the texture detectors. (iii) Deeper layers *saturate*. Over a 30 $\times$ increase in σ , the change grows 17 $\times$ at relu1_2 but only 3.9 $\times$ at relu4_3. Above $\sigma\approx 0.1$ the order reverses: at $\sigma=0.3$, relu4_3 (1.16) and relu3_3 (1.37) are the most robust and relu2_2 (2.42) the least. (iv) Qualitatively, the relu3_3 change follows image structure, while the relu1_2 change looks like the noise itself.

Answer. No feature is noise-invariant, and all of them react more than pixels. Shallow taps behave like a pixel loss and keep growing with σ . Deep taps (relu3_3, relu4_3) are the most robust to moderate and strong noise (relative change 1.16–1.37 at $\sigma=0.3$, vs. 1.96–2.42 for shallow taps) and localize the damage to textured regions, but they are the most sensitive to faint noise. For noisy inputs, our perceptual loss of Sec. 2.2 should therefore weight relu3_3 more than relu2_2.

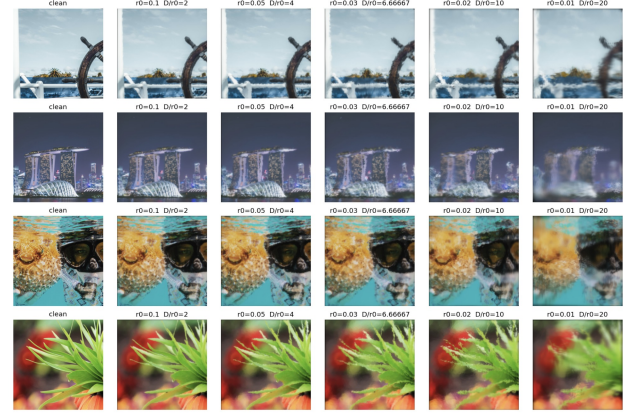


Figure 6. Examples from our turbulence dataset: clean DIV2K crops (left) and their DAATSim versions at the five test strengths (D/r_0 from 2 to 20). Tilt bends straight edges; blur grows with D/r_0 .

2.4. A perceptual feature robust to atmospheric turbulence

Why standard features fail. DAATSim [16] models turbulence as a smooth random *tilt* (each region shifts by a different sub-pixel to few-pixel amount) plus spatially varying *blur*. Sec. 2.2 showed that VGG features are dominated by aliasing under exactly such small, non-stride-aligned shifts. We therefore expect standard perceptual features to treat turbulence as a large change, and we design against that failure mode.

Dataset. We took the 100 diverse high-resolution photos of the DIV2K validation set [1], cut 8 random 256×256 crops from each (800 clean images), and split them by *source photo* into 640 training and 160 test images. Each training crop has 3 turbulent versions, with r_0 drawn uniformly from $[0.02, 0.1]$, i.e. $D/r_0 \in [2, 10]$ (smaller r_0 = stronger turbulence). Each test crop has one version at each of $D/r_0 \in \{2, 4, 6.7, 10, 20\}$. The strongest level, $D/r_0=20$, lies outside the training range and tests generalization (Fig. 6). We wrapped DAATSim’s tilt and blur stages (uniform depth) so that it renders one turbulent frame per image.

Proposed feature. Our feature $z = g(\phi_{AA}(x))$ has three parts:

- Anti-aliased trunk.** A frozen ImageNet VGG-16 in which every max-pool is replaced by max $\rightarrow$ blur $\rightarrow$ subsample (BlurPool [22]). This directly targets the aliasing measured in Sec. 2.2.
- Multi-scale head g .** relu2_2 (BlurPool-downsampled) and relu3_3 are concatenated and passed through two 3×3 conv layers, giving a 128-

```

def dense_infonce(za, zb, n_loc=64, tau=0.1):
    # za, zb: BxDxhwxw unit-norm maps of x and T(x)
    idx = sample_locations(za, n_loc) # same p in both
    qa = gather(za, idx).reshape(-1, D) # (B*n_loc) x D
    qb = gather(zb, idx).reshape(-1, D)
    logits = qa @ qb.t() / tau # positives on diagonal
    t = torch.arange(len(qa), device=za.device)
    ce = F.cross_entropy
    return 0.5 * (ce(logits, t) + ce(logits.t(), t))

```

Code 2. Dense InfoNCE loss on sampled feature locations.

dimensional, unit-normalized feature map at 1/4 resolution (1.18M trainable parameters). The 3×3 receptive field lets the head learn to tolerate local tilt.

3. **Dense contrastive training.** For a clean image x and its turbulent version $T(x)$ we sample 64 locations per image and minimize the symmetric InfoNCE loss [20, 21]:

$$\mathcal{L} = -\frac{1}{|P|} \sum_{p \in P} \log \frac{\exp(z_p^\top z'_p / \tau)}{\sum_{q \in P} \exp(z_p^\top z'_q / \tau)}, \quad (3)$$

where $z = g(x)$, $z' = g(T(x))$, $\tau = 0.1$, and P contains the sampled locations of all images in the batch. The positive pair (same location) teaches invariance to turbulence. The negatives (other locations and other images) prevent the trivial constant solution and keep the feature sensitive to real content changes.

Training uses AdamW (lr 10^{-3} , cosine schedule), batch 16 and 30 epochs. We compare against two ablations: *no anti-aliasing* (original max-pooling) and *invariance-only* (pull positives together without negatives).

Evaluation protocol. A feature that ignores everything is perfectly “invariant” and useless, so invariance alone is not a valid metric. For each test image x we compare $d(x, T(x))$, which should be small, with $d(x, \text{edit}(x))$, where a 64×64 patch is replaced by content from another scene, which should be large. Our main metric is $\text{AUC}_{\text{edit}} = P(d(x, T(x)) < d(x, \text{edit}(x)))$: 1.0 means turbulence is always judged a smaller change than a real edit, and 0.5 is chance. We also report the *distance ratio*, the median $d(x, T(x))$ divided by the median distance between two *different* images (lower = more invariant). Baselines are pixel MSE, blurred pixels (Gaussian, $\sigma=2$), the four VGG taps of Sec. 2.2, an LPIPS-style 4-tap sum [23], and an untrained anti-aliased VGG.

Results. Figs. 7 and 8 and Tab. 4 show four findings. (i) **Standard perceptual features fail completely.** All VGG taps, the LPIPS-style sum and the untrained anti-aliased VGG have $\text{AUC}_{\text{edit}} \approx 0$ at every strength: they always judge even weak turbulence a *larger* change than replacing a whole 64×64 patch. In Fig. 8, `relu3_3` lights

Feature	AUC _{edit} (↑) at $D/r_0 =$					ratio (↓)	
	2	4	6.7	10	20 [†]	2	20 [†]
pixels (MSE)	0.728	0.558	0.388	0.276	0.130	0.036	0.113
blurred pixels	0.998	0.984	0.934	0.817	0.486	0.007	0.052
VGG taps / LPIPS / AA-VGG	≤ 0.001	0.000	0.000	0.000	0.000	0.27–0.34	0.55–0.82
Proposed	0.763	0.218	0.020	0.001	0.000	0.069	0.372
w/o anti-aliasing	0.699	0.190	0.019	0.001	0.000	0.073	0.380
invariance-only	0.042	0.014	0.002	0.000	0.000	0.219	0.634

Table 4. Turbulence results on the 160 test images (from `metrics.csv`). Every feature judges the turbulent copy closer than a *different* image ($\text{AUC} \geq 0.998$, not shown). [†] Stronger than any training turbulence. “VGG taps / LPIPS / AA-VGG” covers the four single VGG taps, the LPIPS-style sum and the untrained anti-aliased VGG, which all behave alike.

up across the whole image under turbulence (rel. dist 0.33 at $D/r_0=4$ vs. 0.11 for the edit). (ii) **The proposed feature is much more robust than VGG.** It reaches $\text{AUC}_{\text{edit}} = 0.76$ at $D/r_0=2$ and lowers the distance ratio $4\text{--}5\times$ (0.07 vs. 0.27–0.34). Its change maps are dark under turbulence and bright on the edited patch; at $D/r_0=4$ it ranks the turbulent copy closer than the edit (0.091 vs. 0.117), which none of the VGG features does. Anti-aliasing helps slightly but consistently (0.76 vs. 0.70), and the contrastive negatives are essential: without them the feature cannot separate edits from turbulence ($\text{AUC} \leq 0.04$). (iii) **But it degrades quickly.** AUC_{edit} falls to 0.22 at $D/r_0=4$ and to ≈ 0 from $D/r_0=6.7$, even inside the training range, and it does not generalize to $D/r_0=20$. (iv) **A blurred-pixel distance is the best feature here** (AUC 1.00 to 0.82 inside the training range, 0.49 at $D/r_0=20$).

Why the problem is hard. The edit changes $64^2/256^2 = 1/16$ of the image, whereas turbulence changes *every* location. For an image-level distance to rank turbulence below the edit, the per-location response to turbulence must be more than about $16\times$ smaller than the response to replaced content. VGG features respond strongly to edges and texture, which is exactly what tilt displaces and blur removes (Sec. 2.2, check 3), so they fail by a wide margin. Our learned head shrinks the turbulence response enough for weak tilt, but its 3×3 receptive field at 1/4 resolution cannot absorb the larger displacements of stronger turbulence. Low-pass filtering wins because it discards precisely the high frequencies that turbulence corrupts, while the edit changes low-frequency color and brightness. Two directions follow from this: computing the learned feature on a low-pass pyramid, and matching features within a local search window (min-distance over small displacements) instead of at fixed coordinates. Two limitations remain: a turbulence-invariant feature is also less sensitive to blur in general, which a restoration loss *should* penalize, and both training and testing use DAATSim in uniform-depth mode, so the sim-to-real gap is untested.

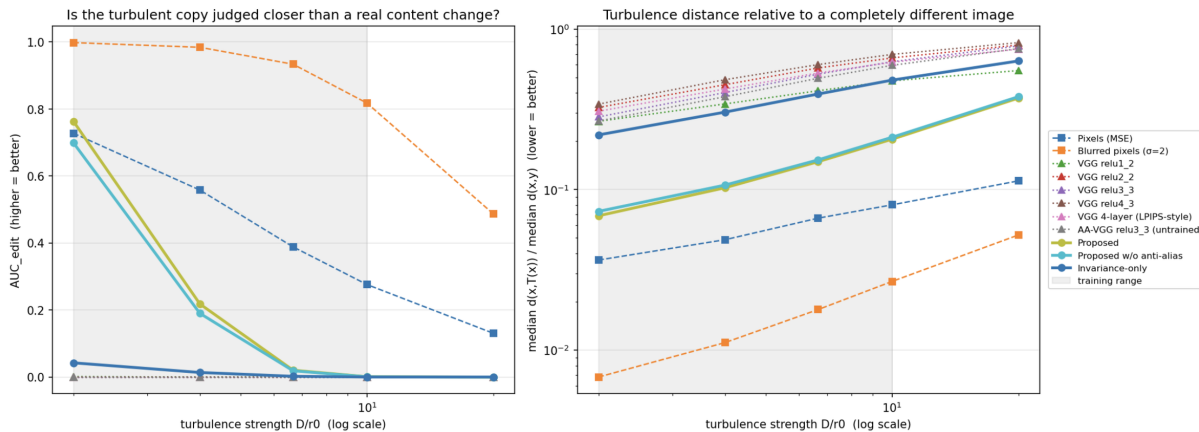


Figure 7. Turbulence robustness on the 160 held-out test images vs. turbulence strength (gray band = training range). Left: AUC_{edit}, the probability that the turbulent copy is judged closer than a 64×64 content edit (higher = better). Right: median turbulence distance divided by the median distance to a different image (log scale, lower = more invariant).

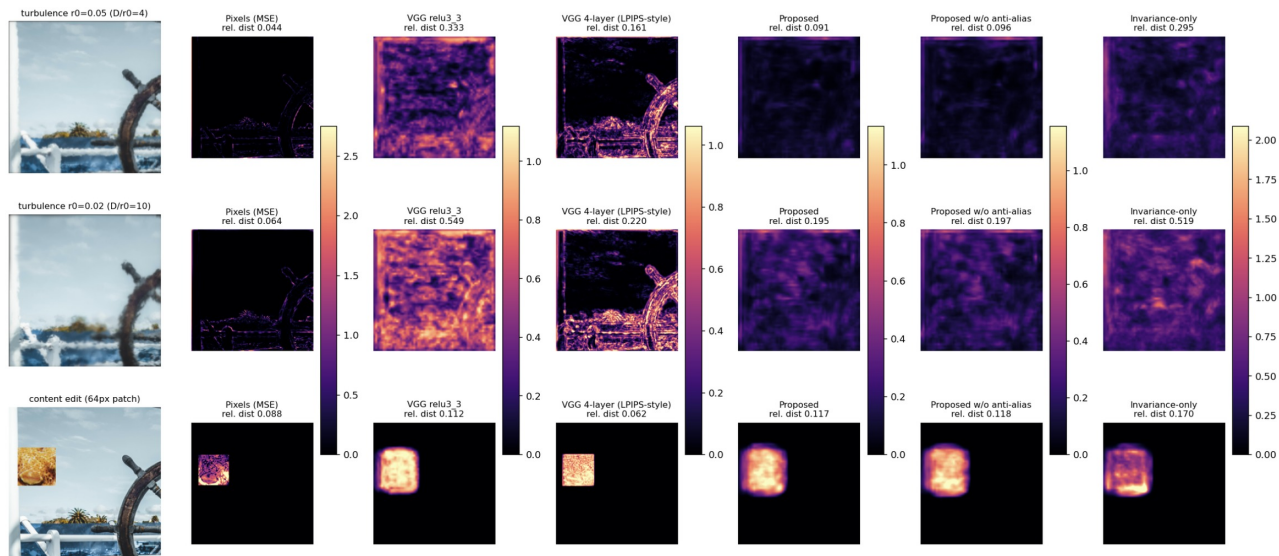


Figure 8. Where each feature changes, for turbulence at $D/r_0=4$ (top) and $D/r_0=10$ (middle) and for a 64×64 content edit (bottom). Values are divided by each feature’s distance to a different image and share one color scale per column; “rel. dist” is the image-level distance. Ideal: dark under turbulence, bright only on the edited patch.

Answer. We built a turbulence-robust feature from an anti-aliased VGG trunk, a multi-scale head and a dense contrastive loss. Under weak turbulence it is far more robust than every standard perceptual feature (AUC_{edit} 0.76 vs. 0.00; 4–5× lower distance ratio). It does not hold up under strong turbulence, where a simple blurred-pixel distance is best. The experiments show that robustness requires suppressing the high-frequency content that VGG features are built on.

3. Problem 2: ViT on the PV image dataset

3.1. Data pre-processing and augmentation

Dataset. We use the RaptorMaps InfraredSolar-Modules dataset [13]: 20,000 single-channel infrared images of 24×40 pixels in 12 classes (11 anomaly types and No-Anomaly). Labels come from `module_metadata.json`. The data is highly imbalanced: No-Anomaly has 10,000 images and Diode-Multi only 175 (Fig. 11). Fig. 9 shows five samples with their labels.

Sample infrared images (24×40 px) with labels; brighter = hotter

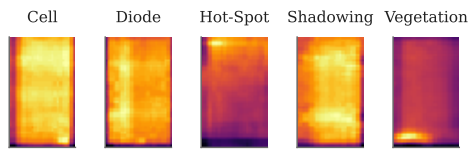


Figure 9. Five samples from the PV dataset with their labels (false color: brighter = hotter).

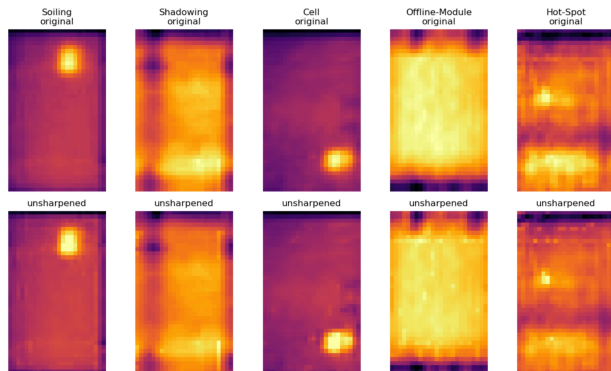


Figure 10. Unsharp masking (radius 1, amount 150%, threshold 2). Top: original. Bottom: sharpened. The defects (hot spots, shadow edges) gain contrast and sharper boundaries.

Unsharp masking. Every image is sharpened with an unsharp mask [5], $\tilde{x} = x + a(x - G_r * x)$, where G_r is a Gaussian blur of radius $r=1$, the amount is $a=1.5$, and differences below 2 gray levels are skipped so that flat regions and noise are not amplified. The radius is 1 rather than the usual 2 because the images are tiny. As Fig. 10 shows, hot spots become brighter with sharper boundaries, and shadow edges and the cell grid become more visible. Faint noise and compression artifacts are also boosted.

Balancing. Classes above 2,500 images (only No-Anomaly) are randomly down-sampled. For every other class all originals are kept, and new images are generated from random unsharpened originals by a random non-empty combination of horizontal flip, vertical flip and color jitter. Because the images are grayscale, the jitter changes only brightness and contrast ($\pm 20\%$). Images are saved as PNG to avoid JPEG artifacts at this size. Tab. 5 summarizes the result: 12,500 originals plus 17,500 augmented images = 30,000. Every augmented image records its source image (`source_id`). The split is done by source image per class (70/15/15%), so an original and all its copies always land in the same split, which avoids train/test leakage. The split has 21,011 / 4,526 / 4,463 images, of which 8,750 / 1,875 / 1,875 are originals.

PV dataset: balancing every class to 2,500 images

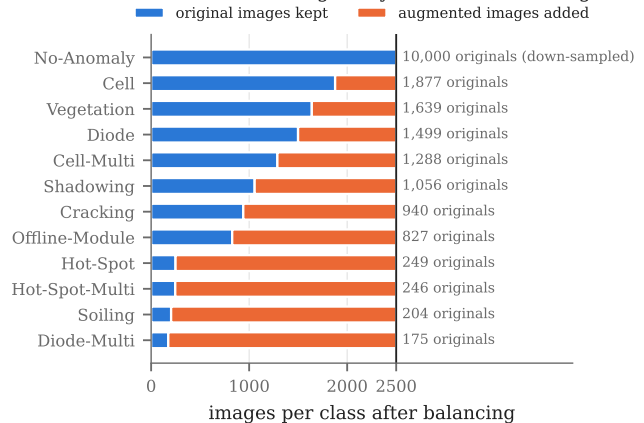


Figure 11. Class balancing to 2,500 images per class. Blue: original images kept. Orange: augmented images added.

Class	Before	Originals kept	Augmented	After
No-Anomaly	10,000	2,500	0	2,500
Cell	1,877	1,877	623	2,500
Vegetation	1,639	1,639	861	2,500
Diode	1,499	1,499	1,001	2,500
Cell-Multi	1,288	1,288	1,212	2,500
Shadowing	1,056	1,056	1,444	2,500
Cracking	940	940	1,560	2,500
Offline-Module	827	827	1,673	2,500
Hot-Spot	249	249	2,251	2,500
Hot-Spot-Multi	246	246	2,254	2,500
Soiling	204	204	2,296	2,500
Diode-Multi	175	175	2,325	2,500
Total	20,000	12,500	17,500	30,000

Table 5. Images per class before and after balancing and augmentation.

```

UNSHARP = ImageFilter.UnsharpMask(radius=1, percent
                                   =150,
                                   threshold=2)
AUGS = {"hflip": ImageOps.mirror, "vflip": ImageOps.
        flip,
        "jitter": T.ColorJitter(.2, .2)} # bright.,
        contr.
for c in classes:
    keep = originals[c]
    if len(keep) > 2500: # down-
        sample
        keep = rng.sample(keep, 2500)
    save([UNSHARP(load(p)) for p in keep])
    for _ in range(2500 - len(keep)): # up-sample
        ops = rng.choice(non_empty_subsets(AUGS))
        src = UNSHARP(load(rng.choice(keep)))
        save(apply(ops, src)) # e.g. hflip+jitter

```

Code 3. Unsharp masking and augmentation (excerpt).

3.2. ViT-B/32 fine-tuning strategies

We fine-tune the ImageNet-1k pretrained ViT-B/32 [4] (torchvision), replacing its head with a 12-way linear layer. The 24×40 images are replicated to 3 channels, resized to

```

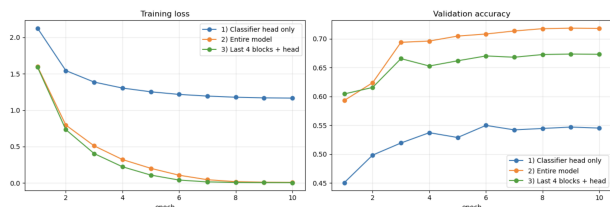
m = vit_b_32(weights=ViT_B_32_Weights.IMAGENET1K_V1)
m.heads.head = nn.Linear(768, 12)
for p in m.parameters(): p.requires_grad = False
parts = {"head": [m.heads], "full": [m],
        "last": [m.heads, m.encoder.ln,
                 *m.encoder.layers[-4:]]}[strategy]
for part in parts:
    for p in part.parameters(): p.requires_grad = True

```

Code 4. Selecting the trainable parameters for each strategy.

Strategy	Trainable	Train	Val	Test	Test (orig.)	Time (min)
1) Head only	9.2K	60.76	54.97	56.31	55.15	1.3
2) Entire model	87.5M	99.94	71.83	73.25	76.37	4.2
3) Last 4 blocks + head	28.4M	99.98	67.34	68.70	72.48	2.3

Table 6. ViT-B/32 on the balanced PV dataset (accuracy in %, 10 epochs, best validation epoch). “Test (orig.)” = the 1,875 non-augmented test images. Time = training time on one A100 MIG slice.

Figure 12. ViT-B/32 learning curves. Left: training loss vs. epoch. Right: validation accuracy (fraction) vs. epoch. Head-only training plateaus at a high loss; the two strategies that adapt transformer blocks drive the training loss to ≈ 0 by epoch 8.

224 $\times$ 224 (bilinear, on the GPU) and ImageNet-normalized, so each image becomes 49 patches of 32 $\times$ 32. All strategies use AdamW (weight decay 0.01), a one-cycle schedule with 10% warm-up, batch 128, bfloat16 and 10 epochs, and we keep the epoch with the best validation accuracy. The learning rate is chosen per strategy, because a randomly initialized head needs a larger step than pretrained blocks:

1. **Head only:** 9,228 trainable parameters (0.01%), lr 10^{-3} .
2. **Entire model:** 87.46M parameters (100%), lr $5 \cdot 10^{-5}$.
3. **Last 4 blocks + final LayerNorm + head:** 28.36M parameters (32.4%), lr 10^{-4} .

3.3. Analysis

Accuracy. Fine-tuning the **entire model** gives the best accuracy (73.25% test, 76.37% on original images), 4.6 points above the **last 4 blocks** (68.70%) and 16.9 points above the **head only** (56.31%) (Tab. 6, Fig. 12). Validation and test agree within 1.5 points for every strategy, so the ranking is reliable.

Training efficiency. Training time grows with the depth of back-propagation: 1.3 min (head only), 2.3 min (last 4 of 12 blocks) and 4.2 min (all blocks). All three strategies run the same full forward pass, so the time grows less than the trainable-parameter count (9.2K, 28.4M and 87.5M). Optimizer memory, however, scales with the trainable parameters.

Why. (i) *Domain gap.* The images are upsampled 24 $\times$ 40 single-channel thermal maps, nothing like ImageNet photos. A linear head on frozen ImageNet features *underfits*: training accuracy is only 60.8% and the loss plateaus at ≈ 1.2 , so the frozen representation does not separate the PV classes. (ii) *Where to adapt.* Unfreezing the last 4 blocks lets the high-level features specialize, and the model fits the training set ($\approx 100\%$). Full fine-tuning additionally adapts the patch embedding and early blocks to the blurry, low-contrast thermal patterns, which gives a further 4.5–4.6 points on validation and test. Since the first layers are where ImageNet and thermal images differ most, their adaptation matters. (iii) *Overfitting.* Both unfrozen variants reach $\approx 100\%$ training accuracy while validation plateaus after epoch 3–6. They memorize the training set, especially the small classes, whose 2,500 images are mostly augmented copies of 175–250 originals. The best-validation checkpoint limits the damage. (iv) *Hard classes.* The remaining errors are mostly counting problems (Cell vs. Cell-Multi, Hot-Spot vs. Hot-Spot-Multi) and Soiling (204 originals), as the ResNet-50 reference below shows.

Answer. Full fine-tuning is the most accurate (73.3% test) and, at 4.2 min, still cheap on this small dataset. It is the best choice when accuracy matters. Fine-tuning the last 4 blocks is the best *trade-off* when compute or memory is limited: 94% of the full model’s accuracy in 55% of the time with 32% of the trainable parameters. Training only the head is fastest but not usable (56.3%), because frozen ImageNet features do not transfer to 24 $\times$ 40 thermal images.

CNN reference (ResNet-50, full fine-tuning). To calibrate the ViT numbers we also fine-tuned all layers of an ImageNet ResNet-50 on the same split (Tab. 7). The first version (AdamW, lr 10^{-4} , 224 $\times$ 224 inputs) overfit after epoch 3: train accuracy reached 98% while validation accuracy peaked at 63.9%. Keeping the 24 $\times$ 40 aspect ratio (224 $\times$ 128 inputs) and adding online flips and shifts, label smoothing 0.1, weight decay 0.05 and dropout 0.2 raised validation accuracy to 69.3% and test accuracy from 67.2% to 70.8%. Test-time augmentation (averaging the original, h-flip and v-flip predictions) raised test accuracy to 72.6%. The fully fine-tuned ViT-B/32 (73.3%) is slightly better than the best ResNet-50, even without test-time augmentation. Per-class test accuracy of the improved model ranges from 96.3% (Diode-Multi) and 92.1% (Diode) down

ResNet-50 (all layers trained)	Test	Test (originals)
baseline, 224×224 , 10 epochs	67.2	69.8
improved, 224×128 , 15 epochs	70.8	73.6
+ test-time augmentation	72.6	75.7
ViT-B/32, entire model (Tab. 6)	73.3	76.4

Table 7. CNN reference on the PV split (accuracy in %). “Originals” = the 1,875 non-augmented test images.

Classes (sampled every 6.5-th sorted WordNet ID)	100 of 650
Images per class (train / val / test)	300 (240 / 30 / 30)
Total images (train / val / test)	30,000 (24,000 / 3,000 / 3,000)
Fraction of the ImageNet folder on Sol	3.6%
Median image size (range)	500×375 (W 62–3648, H 38–2851)
Color modes in a 3,000-image sample	RGB 98%, grayscale 2%
RGB mean (ImageNet reference)	0.500, 0.467, 0.427 (0.485, 0.456, 0.406)
RGB std (ImageNet reference)	0.281, 0.278, 0.295 (0.229, 0.224, 0.225)

Table 8. Prototype dataset statistics.

to 56.8% (Hot-Spot-Multi), 50.4% (Cell-Multi) and 44.7% (Soiling). The largest confusions are Cell-Multi→Cell (80), Soiling→Cracking (78) and Hot-Spot-Multi→Hot-Spot (67).

4. Problem 3: Custom ResNet on ImageNet

4.1. Prototype dataset

Full split. The ImageNet training folder on Sol contains **650** class folders (not 1,000) with 832,571 images: at least 889 and at most 1,300 per class, with a median of 1,300. Class names come from the standard WordNet-ID→name index. Because only this folder has labels, we split *each class* 80/10/10 into train/val/test: 666,057 / 83,257 / 83,257 images. No image files are copied. The splits are CSV manifests (path, WordNet ID, label, split).

Prototype. We take **100 classes**, every 6.5-th WordNet ID in sorted order. Sorted IDs roughly group related concepts, so even spacing covers all super-categories instead of, for example, 100 dog breeds. We take **300 images per class** (240 train / 30 val / 30 test), sampled from the corresponding full splits, so every prototype test image is also a full-dataset test image. This gives 30,000 images (3.6% of the data) with 24,000 / 3,000 / 3,000 in train/val/test. Tab. 8 lists the statistics. A 30-epoch ResNet run on the prototype takes about 5 minutes on one A100, which makes design iteration fast. At the same time 240 images per class are enough to rank models, although the run-to-run spread is about ± 1 point (Sec. 4.2).

4.2. Custom ResNet-36

Design. ResNet-34 [7] stacks [3, 4, 6, 3] BasicBlocks (two 3×3 convs each) in four stages: 1 stem conv + 32 block convs + 1 FC = 34 weighted layers. Our ResNet-36 uses

Stage	Output	Channels	ResNet-34	ResNet-36
stem: 7×7 conv /2, max-pool /2	56×56	64	1 conv	1 conv
stage 1	56×56	64	3 blocks	3 blocks
stage 2	28×28	128	4 blocks	4 blocks
stage 3	14×14	256	6 blocks	7 blocks
stage 4	7×7	512	3 blocks	3 blocks
head: avg-pool, FC	1×1	–	1 FC	1 FC
weighted layers / params (100 cl.)			34 / 21.34M	36 / 22.52M

Table 9. ResNet-34 vs. our ResNet-36 for a 224×224 input. A block is two 3×3 conv layers with a skip connection.

	Ep.	Train	Val top-1/5	Test top-1/5	Gap	Time
ResNet-34	30	57.4	52.7 / 77.5	53.5 / 79.0	−3.9	5.1
ResNet-36	30	57.6	53.5 / 77.6	53.4 / 78.7	−4.2	5.1
ResNet-34	100	91.8	64.1 / 83.1	62.5 / 83.2	29.3	18.5
			$\pm 0.0 / \pm 0.1$	$\pm 0.0 / \pm 0.1$		
ResNet-36	100	91.8	63.2 / 82.9	63.1 / 82.6	28.8	18.4
			$\pm 0.0 / \pm 0.2$	$\pm 1.0 / \pm 0.2$		

Table 10. Depth comparison on the prototype (%). 30-epoch runs: one seed. 100-epoch runs: mean $\pm$ s.e. over seeds 0 and 1. Gap = train – test top-1. Time = training minutes on one A100.

[3, 4, 7, 3]: one extra block in stage 3 (Tab. 9), giving 36 layers. 1×1 shortcut convs are not counted, as in [7]. The extra block adds 1.18M parameters (21.34M→22.52M with a 100-class head) and about 0.23 GMACs, roughly 6% of ResNet-34’s compute. All weights are trained from scratch with Kaiming initialization, and the last BatchNorm of each block is zero-initialized so that every block starts as an identity map.

Expected benefit. Stage 3 works at 14×14 with 256 channels, where the network forms object-part representations. Deeper ResNets add depth mainly here (ResNet-101 has 23 stage-3 blocks), because it is the cheapest place to add capacity: each extra block costs the same compute as in other stages but has fewer parameters than a stage-4 block. We hoped the extra block would give a larger receptive field and one more non-linearity for composing parts. Residual connections should let the network ignore the block (identity map) if it does not help.

Training recipe (all prototype runs). SGD with Nesterov momentum 0.9, lr 0.05 (0.1 per 256 images, linearly scaled to batch 128 [6]), 2-epoch linear warm-up followed by cosine decay [12], weight decay $5 \cdot 10^{-4}$ (not applied to BatchNorm parameters and biases), label smoothing 0.1 [18], RandomResizedCrop $224 +$ horizontal flip, batch 128 on one A100, and bfloat16. The model with the best validation top-1 is evaluated once on the test split.

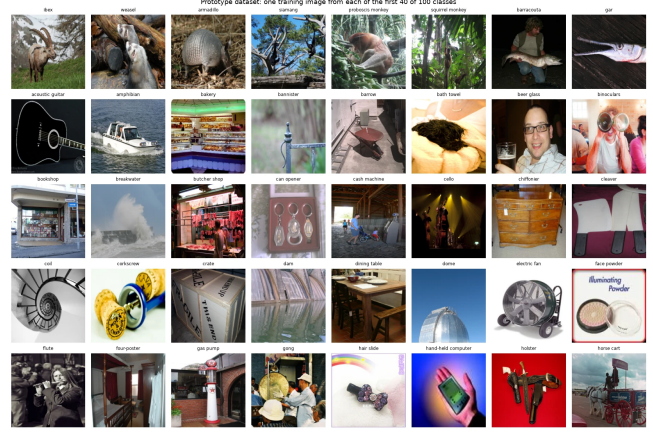
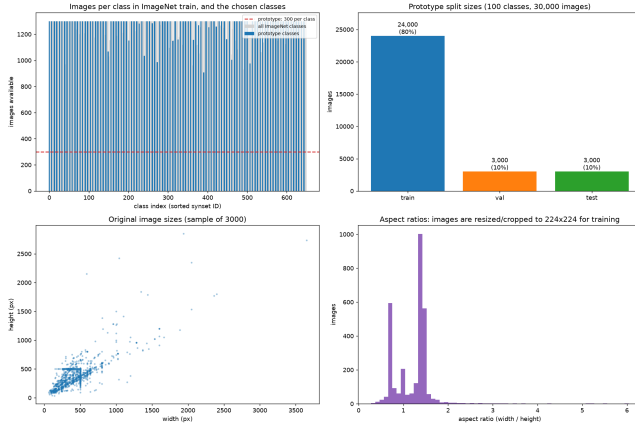


Figure 13. Prototype dataset. Left: images per class in Sol’s ImageNet folder with the 100 selected classes (blue), split sizes, original image sizes and aspect ratios (sample of 3,000). Right: one training image from each of the first 40 prototype classes.

Results. Tab. 10 and Fig. 14 show that the two models are statistically indistinguishable. After 30 epochs they differ by 0.04 points in test top-1. After 100 epochs ResNet-36 is 0.6 points better on test top-1 but 0.9 points worse on validation. Both differences are smaller than the seed-to-seed variation: ResNet-36’s test top-1 moved from 62.1% to 64.0% between seeds. The extra layers did not hurt optimization, since training accuracy is equal. Both models overfit heavily (about 92% train vs. 62–63% test), so on 240 images per class the bottleneck is data, not capacity, and one more block cannot be exploited.

Answer. ResNet-36 matches ResNet-34 on the prototype (test top-1 63.1% vs. 62.5%, within the ± 1 -point seed noise) at +6% compute. The benefit we hoped for, more part-level capacity in stage 3, cannot appear while both models are data-limited. We therefore keep ResNet-36 as the base model and test it at full scale (Sec. 4.4).

4.3. Custom activation: ArcGate

Definition. We propose

$$\text{ArcGate}(x) = x \left(\frac{1}{2} + \frac{1}{\pi} \arctan x \right), \quad (4)$$

$$f'(x) = \frac{1}{2} + \frac{\arctan x}{\pi} + \frac{x}{\pi(1+x^2)}. \quad (5)$$

The gate $\frac{1}{2} + \frac{1}{\pi} \arctan x$ is the CDF of the standard *Cauchy* distribution. ArcGate is thus the heavy-tailed member of the self-gating family $x \Phi(x)$ that includes GELU (Gaussian CDF) [8] and SiLU/Swish (logistic CDF) [15]. It is not available in PyTorch. Its properties (Fig. 15):

- $f(0) = 0$, $f'(0) = \frac{1}{2}$, and $f(x) \approx x - \frac{1}{\pi}$ for large positive x , so it behaves like ReLU for strong activations.
- It is monotonic and bounded below by $-1/\pi \approx -0.318$, approached as $x \rightarrow -\infty$ (expansion: $f(x) = -\frac{1}{\pi} +$

Prototype (100 classes), 100 epochs; dots = the two seeds

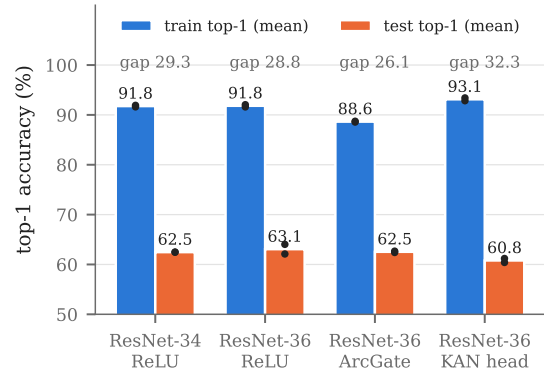


Figure 14. Train and test top-1 on the prototype after 100 epochs (bars: mean of 2 seeds; dots: individual seeds). ArcGate has the smallest train–test gap; the KAN head has the largest.

$\frac{1}{3\pi x^2} + \dots$). Unlike SiLU and GELU, it has no negative dip.

- For negative inputs the gradient decays only polynomially, $f'(x) \approx \frac{2}{3\pi|x|^3}$, instead of exponentially (SiLU/GELU) or to exactly 0 (ReLU). Units therefore never “die”, and negative pre-activations still receive a learning signal.

Expectations. Compared with ReLU we expected (i) smoother optimization, because the gradient is continuous at 0; (ii) no dead units; and (iii) a mild regularization effect, because the soft gate passes small negative values and the network cannot easily form the sharp, sparse codes that support memorization. We expected similar or slightly better accuracy, at a small compute cost for arctan.

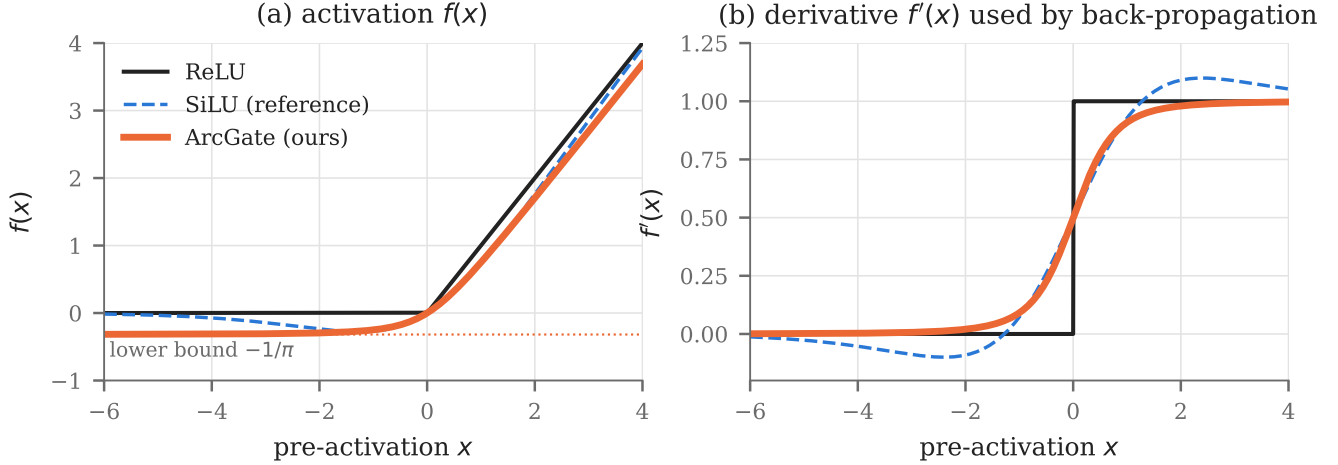


Figure 15. ArcGate compared with ReLU (and SiLU for reference). (a) ArcGate is smooth, monotonic, passes small negative values and is bounded below by $-1/\pi$. (b) Its derivative rises smoothly from 0 to 1 and, unlike ReLU's, is never exactly zero.

```
class _ArcGateFn(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return x * (0.5 + torch.atan(x) / math.pi)
    @staticmethod
    def backward(ctx, g):
        (x,) = ctx.saved_tensors
        return g * (0.5 + torch.atan(x) / math.pi
                    + x / (math.pi * (1 + x * x)))
```

Code 5. ArcGate with a hand-written backward pass.

ResNet-36, 100 ep.	Train	Val top-1/5	Test top-1/5	Gap	Time
ReLU	91.8	63.2 / 82.9	63.1 / 82.6	28.8	18.4
ArcGate	88.6	63.2 / 83.5	62.5 / 83.6	26.1	20.3
ReLU + KAN head	93.1	60.7 / 80.1	60.8 / 80.8	32.3	17.9

Table 11. Activation comparison on the prototype (% , mean $\pm$ s.e. over 2 seeds). Gap = train – test top-1. Time = minutes per 100-epoch run (seed 0).

Implementation. ArcGate replaces every ReLU in ResNet-36. We use a custom `autograd.Function` that stores only the input x and recomputes $f'(x)$ in the backward pass (Listing 5). This keeps activation memory equal to ReLU's.

Results. Tab. 11 shows that ArcGate matches ReLU in top-1: validation accuracy is identical (63.2%), and the test difference (-0.5) is within ReLU's own ± 1.0 seed variation. ArcGate improves top-5 accuracy in both seeds and on both evaluation sets ($+0.6$ validation, $+1.1$ test). Its most robust effect is *less overfitting*. It reaches the same test accu-

racy with about 3 points lower training accuracy (88.6% vs. 91.8%), which shrinks the train–test gap from 28.8 to 26.1 points; the reduction appears in both seeds (25.9 and 26.3 vs. 29.5 and 28.0). This is the regularizing effect we expected from a soft gate that never blocks the gradient completely. The cost is 10% longer training (20.3 vs. 18.4 min), from the arctan computation.

Extra: KAN classifier head. We also replaced the final FC layer with a Kolmogorov–Arnold layer [11]. Each input–output pair gets a learnable function $w \text{SiLU}(x) + \sum_k c_k B_k(x)$, where B_k are cubic B-splines on a 5-interval grid over $[-3, 3]$, preceded by a LayerNorm. The head adds 0.41M parameters. It was consistently worse (-2.3 test top-1, -1.8 top-5) and overfit the most (gap 32.3), because a more flexible head memorizes the 240 images per class more easily.

Answer. ArcGate is a new Cauchy-gated activation. It matches ReLU in top-1 accuracy, gives a small but consistent top-5 gain ($+1.1$ test points), and reduces overfitting by 2.7 points in both seeds, at 10% extra training time. We use ResNet-36 + ArcGate for the full-dataset runs.

4.4. Full ImageNet on GPUs and Gaudi accelerators

Setup. ResNet-36 + ArcGate (22.80M parameters, 650-way head) is trained from scratch on the full split (666,057 training images) for 10 epochs with distributed data-parallel training on two devices, in two separate runs: (i) 2×NVIDIA A100-SXM4-80GB with NCCL, and (ii) 2×Intel Gaudi 2 (HL-225) with HCCL in *lazy mode* [9]. Lazy mode collects operations into a graph and executes it at each `htcore.mark_step()` (Listing 6). Both runs use identical hyperparameters (Tab. 12) and the same split and seed, so the platforms are directly comparable. JPEG

Model	ResNet-36 [3,4,7,3], ArcGate, 22.80M params
Data	650 classes; 666,057 / 83,257 / 83,257 images
Epochs	10
Batch	128 per device $\times$ 2 devices = 256
Optimizer	SGD, Nesterov momentum 0.9
Learning rate	0.1, 1-epoch linear warm-up, cosine decay to 0
Weight decay	10^{-4} (none on BatchNorm and biases)
Regularization	label smoothing 0.1
Augmentation	RandomResizedCrop 224, horizontal flip
Evaluation	resize 256, center crop 224
Precision	bfloat16 autocast

Table 12. Hyperparameters of the full-ImageNet runs (identical on A100 and Gaudi 2).

```
import habana_frameworks.torch.core as htcore
dist.init_process_group("hccl")
model = DDP(model.to("hpu"), bucket_cap_mb=100,
              gradient_as_bucket_view=True)
loss.backward(); htcore.mark_step() # run the graph
opt.step(); htcore.mark_step()
# SBATCH: -p gaudi -q class_gaudi --gres=gpu:h1225:2
# export PT_HPU_LAZY_MODE=1
# torchrun --nproc_per_node 2 train_imagenet.py ...
```

Code 6. Gaudi-specific parts of the training loop and job script (excerpt).

	2×A100	2×Gaudi 2
Train top-1 (augmented images)	54.43	54.33
Val top-1 / top-5	59.97 / 82.56	59.75 / 82.32
Test top-1 / top-5	60.38 / 82.87	60.28 / 82.76
Best epoch	10 / 10	10 / 10
Training time, 10 epochs	53.9 min	46.8 min
Whole job (incl. test + latency)	63.1 min	52.6 min
Training throughput	2,121 img/s	2,381 img/s
Latency, batch 1 (mean / p50 / p90)	6.21 / 6.19 / 6.28 ms	9.15 / 9.15 / 9.19 ms
Latency, batch 64 (mean / p50 / p90)	7.72 / 7.65 / 8.18 ms	9.35 / 9.12 / 9.18 ms
Inference throughput, batch 64	8,286 img/s	6,844 img/s

Table 13. Full-ImageNet results for ResNet-36 + ArcGate (accuracy in %). Latency is measured on one device with bfloat16 and a 224×224 input, after 20 warm-up iterations, over 100 timed forward passes, excluding data loading.

decoding uses 14 loader processes per device. The model is selected on the validation split and evaluated *once* on the 83,257-image test split.

Results. Tab. 13 and Figs. 16 and 17 summarize the results. *Accuracy.* Both platforms reach the same accuracy: test top-1 differs by 0.10 points and top-5 by 0.11 points, which is within run-to-run noise (data order, bfloat16 rounding). The model therefore transfers to Gaudi without loss. With $28\times$ more training images (and $6.5\times$ more classes) than the prototype, it reaches 60.4% top-1 in only 10 epochs. The best epoch is the last one, so longer training would improve accuracy further. *Training speed.* Gaudi 2 finished the 10 epochs 13% faster (46.8 vs. 53.9 min, 2,381

ResNet-36 + ArcGate, full ImageNet (650 classes), 2×A100

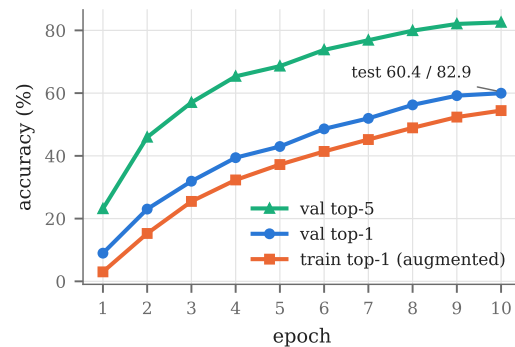


Figure 16. Full-ImageNet training curve (A100 run). Validation accuracy is still rising at epoch 10, and train accuracy (measured on augmented crops with label smoothing) stays below validation accuracy, so the model is not overfitting.

vs. 2,121 img/s). Training at this scale is partly limited by CPU JPEG decoding, so part of the gap may come from the Gaudi node's CPUs and storage rather than from the accelerator itself. *Latency.* The A100 is faster at inference: 6.2 vs. 9.2 ms at batch 1 and 8,286 vs. 6,844 img/s at batch 64. On Gaudi the batch-1 and batch-64 latencies are almost equal (9.15 vs. 9.12 ms median). This indicates a fixed per-call cost of about 9 ms for graph launch in lazy mode, which dominates small batches. Gaudi throughput scales $63\times$ from batch 1 to batch 64 at nearly constant latency.

Answer. ResNet-36 + ArcGate was trained for 10 epochs on full ImageNet (650 classes). It reaches 60.4% / 82.9% test top-1 / top-5 on A100 and 60.3% / 82.8% on Gaudi 2. Training took 53.9 min on 2×A100 and 46.8 min on 2×Gaudi 2. Batch-1 latency is 6.2 ms (A100) and 9.2 ms (Gaudi 2).

4.5. Vision transformer vs. custom ResNet

Model and recipe. We implement ViT-S/16 [4, 19]: 16×16 patches (196 tokens + class token), 12 transformer blocks, 6 heads, width 384, MLP width 1,536, and 21.9M parameters. This size was chosen to match ResNet-36 (22.8M). It is trained from scratch with the standard ViT recipe: AdamW (lr $1.25 \cdot 10^{-4}$ for batch 128, i.e. $5 \cdot 10^{-4}$ per 512 images, linearly scaled), weight decay 0.05, gradient clipping 1.0, warm-up (5 epochs on the prototype, 1 on the full set) followed by cosine decay, and label smoothing 0.1. Data, augmentation, epochs and seed are identical to the ResNet runs. It ran on one A100 with batch 128.

Results. The ViT does *not* outperform our ResNet (Tab. 14, Fig. 18). On the prototype both models fit the training set equally well (88.6% vs. 87.5%), but the ViT reaches only 35.6% test top-1 against 62.7%. Its generalization gap is 52 points and its best validation epoch is 62 of 100. With 650 classes and 666k images the gap shrinks

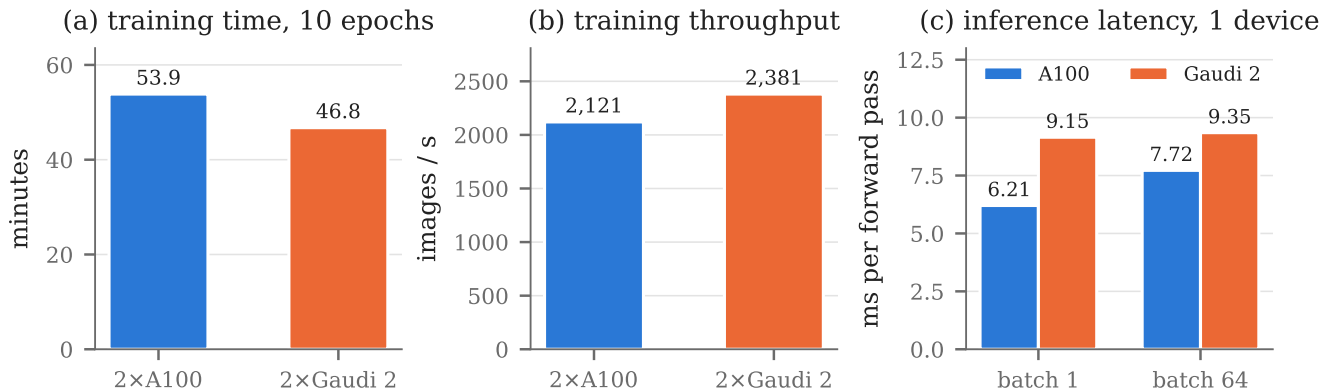


Figure 17. A100 vs. Gaudi 2 with identical model, data and hyperparameters. Gaudi 2 trains 13% faster, while the A100 has lower single-device inference latency at both batch sizes.

	ResNet-36 +ArcGate	ViT-S/16 (ours)
Parameters (100 / 650 classes)	22.5 / 22.8M	21.7 / 21.9M
Depth	36 layers	12 blocks
Prototype 100 ep.	Train top-1	88.6
	Val top-1 / top-5	62.8 / 83.3
	Test top-1 / top-5	62.7 / 83.7
	Train time (1 A100)	20.3 min
Full ImageNet 10 ep.	Train top-1	54.4
	Val top-1 / top-5	60.0 / 82.6
	Test top-1 / top-5	60.4 / 82.9
	Train time (devices)	53.9 min (2)
	Latency, batch 1	6.21 ms
	Latency, batch 64	7.72 ms

Table 14. Custom ResNet vs. ViT trained from scratch (seed 0, accuracy in %, latency on one A100 in bfloat16).

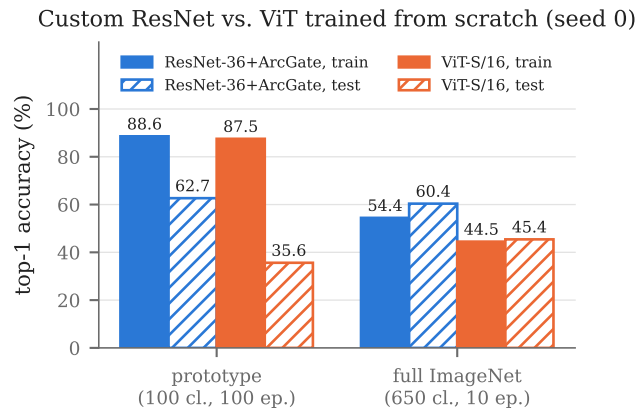


Figure 18. Train (solid) and test (hatched) top-1 for ResNet-36+ArcGate and ViT-S/16. On the prototype the ViT fits the training set as well as the ResNet but generalizes poorly. With 28× more data the gap shrinks and the ViT no longer overfits.

from 27 to 15 points (60.4% vs. 45.4%), and the ViT no longer overfits (train 44.5% < test 45.4%). Its best epoch is also the last, so it is underfitting. The explanation is inductive bias: convolutions build in locality and translation equivariance, while a ViT must learn from data that neighboring patches are related. This requires far more data or longer training: the original ViT pre-trained on 14–300M images, and DeiT used 300 epochs with strong augmentation [4, 19]. For a single image the ViT is 8% faster (5.74 vs. 6.21 ms). It runs 12 wide blocks, while the ResNet runs 36 sequential layers, which adds launch overhead. At batch 64 the ResNet is 36% faster, because attention and MLPs over 197 tokens cost more compute than the ResNet’s convolutions. Note that the full-data ViT trained on one GPU (batch 128) and the ResNet on two (batch 256), so their training times are not directly comparable.

Answer. No. At matched size (~22M parameters) and an identical 10-epoch budget, ResNet-36 + ArcGate beats ViT-S/16 by 15.0 points in top-1 (60.4% vs. 45.4%) and 12.9 points in top-5, and has 36% higher batch-64 throughput. The ViT is slightly faster at batch 1, and its deficit shrinks as data grows (27 → 15 points).

5. Conclusion

Pixel losses and raw VGG features disagree about what a “small” change is. Much of VGG’s sensitivity to imperceptible shifts comes from aliasing, which motivated the anti-aliased, contrastively trained turbulence feature. On PV infrared data, class balancing with grouped (leakage-free) splits is essential, and fine-tuning strategies trade accuracy against compute. On ImageNet, a soft Cauchy-gated activation regularizes as well as it fits. The same model trains to identical accuracy on A100 and Gaudi 2, faster on Gaudi. Convolutional inductive bias remains decisive against a ViT trained from scratch at this data and compute budget.

Acknowledgements

Data and code. RaptorMaps InfraredSolarModules dataset [13]; DIV2K [1]; DAATSim simulator (<https://github.com/Riponcs/DAATSim>) [16]; ImageNet [3] as provided on Sol; PyTorch and torchvision [14] (pre-trained VGG-16, ViT-B/32, ResNet-50); the KAN layer follows [11]. **Documentation.** ASU Research Computing Sol and SBATCH documentation [2]; the course's Gaudi example script (Lab 14); Intel Gaudi documentation [9]; the CVPR 2026 author kit for this template. **Compute.** ASU Research Computing (Sol A100 GPUs and Gaudi 2 nodes); the instructor granted the `class_gaudi` QOS. **AI assistance.** Claude (Anthropic), an AI assistant, was used to help write and debug code and Slurm job scripts and to draft this report. All experiments were run, and all numbers produced, by the author on Sol. **Classmates.** [TODO: list any classmate help, or write "None".]

References

- [1] Eirikur Agustsson and Radu Timofte. NTIRE 2017 challenge on single image super-resolution: Dataset and study. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017. 4, 14
- [2] ASU Research Computing. Sol supercomputer documentation: Slurm SBATCH job scripts. <https://asurc.atlassian.net/wiki/spaces/RC>, 2025. 1, 14
- [3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009. 14
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. 7, 12, 13
- [5] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018. 7
- [6] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large mini-batch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017. 9
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 9
- [8] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. 10
- [9] Intel Corporation. Intel Gaudi documentation: PyTorch lazy mode and distributed training with HCCL. <https://docs.habana.ai>, 2025. 11, 14
- [10] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *European Conference on Computer Vision (ECCV)*, 2016. 2
- [11] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024. 11, 14
- [12] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. 9
- [13] Matthew Millendorf, Edward Obropta, and Nikhil Vadhavkar. Infrared solar module dataset for anomaly detection. In *ICLR 2020 Workshop on AI for Earth Sciences*, 2020. 6, 14
- [14] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 1, 14
- [15] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017. 10
- [16] Ripon Kumar Saha, Yufan Zhang, Jinwei Ye, and Suren Jayasuriya. DAATSim: Depth-aware atmospheric turbulence simulation for fast image rendering. In *Computer Graphics Forum*, page e70241, 2025. 4, 14
- [17] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015. 2
- [18] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 9
- [19] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, 2021. 12, 13
- [20] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 5
- [21] Xinlong Wang, Rufeng Zhang, Chunhua Shen, Tao Kong, and Lei Li. Dense contrastive learning for self-supervised visual pre-training. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 5
- [22] Richard Zhang. Making convolutional networks shift-invariant again. In *International Conference on Machine Learning (ICML)*, 2019. 2, 4
- [23] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 2, 5